

LEOPARDO RH

Plateforme SaaS de Gestion RH & Opérationnelle

Module Finance & Gestion

Factures · Dépenses · Achats · Trésorerie · App Mobile

Dossier Complet Phase 1 Finance + Roadmap Post-Phase

Périmètre :	Factures · Dépenses · Achats · Trésorerie · Dashboard financier
Rôles :	Manager (lecture globale) + Gestionnaire financier (saisie complète)
Surfaces :	App Flutter (mobile-first) + Dashboard Blade (web)
Effort :	~42 jours-développeur · 7 semaines · 1 développeur
Sprints :	6 sprints (A → F) + Roadmap post-phase documentée
Plan cible :	Business (fonctions de base) · Enterprise (complet + analytics)

TABLE DES MATIÈRES

1.	Résumé Exécutif	Vision produit, métriques clés, alignement plateforme
2.	L'Opportunité & Positionnement	Marché, valeur, concurrents, impact tarifaire
2.1.	Contexte et problème client	
2.2.	Proposition de valeur	
2.3.	Rôle Gestionnaire financier	Le nouveau rôle au cœur du module
2.4.	Impact modèle économique	
2.5.	Analyse risques	
3.	Architecture Technique	Stack, flux, composants, sécurité
3.1.	Vue d'ensemble du système	
3.2.	Flux de données financières	
3.3.	Sécurité et isolation des données	
3.4.	Génération PDF (factures & rapports)	
4.	Modèle de Données Complet	9 tables détaillées colonne par colonne
4.1.	Table invoices (factures clients)	
4.2.	Table invoice_items	
4.3.	Table expenses (dépenses)	
4.4.	Table expense_categories	
4.5.	Table purchase_orders (achats)	
4.6.	Table purchase_items	
4.7.	Table suppliers (fournisseurs)	
4.8.	Table finance_accounts (trésorerie)	
4.9.	Table finance_transactions	
4.10.	Modifications tables existantes	
5.	RBAC — Matrice des Permissions	Manager vs Gestionnaire vs Employé
6.	Écrans Mobile — Spécifications complètes	18 écrans Flutter détaillés
6.1.	Navigation et structure	
6.2.	Écrans Manager	
6.3.	Écrans Gestionnaire financier	
6.4.	Flux de création de facture (détail)	
7.	Contrats API Complets	Tous les endpoints · schemas · erreurs
7.1.	Endpoints Factures	
7.2.	Endpoints Dépenses	

7.3.	Endpoints Achats & Fournisseurs	
7.4.	Endpoints Trésorerie & Rapports	
8.	Génération PDF Financiers	DomPDF · templates · envoi email
9.	Plan d'Implémentation — 65 Tâches	Sprint A→F avec priorités et dépendances
Sprint A	Infrastructure & Base de données	Sem. 1 · 14 tâches
Sprint B	API Backend — Documents	Sem. 2 · 12 tâches
Sprint C	API Backend — Trésorerie & PDF	Sem. 3 · 10 tâches
Sprint D	Dashboard Web Blade	Sem. 4 · 10 tâches
Sprint E	Application Flutter Mobile	Sem. 5-6 · 13 tâches
Sprint F	Tests, Sécurité & Déploiement	Sem. 7 · 6 tâches
10.	Tests — Stratégie Complète	Coverage, cas critiques, E2E
11.	Roadmap Post-Phase 1	Phases 2→5 spécifiées avec tâches
Phase 2	Analytiques & Tableaux de bord IA	
Phase 3	Intégration bancaire & rapprochement	
Phase 4	Module Paie intégrée complète	
Phase 5	Écosystème & API publique	
12.	Définition of Done — 18 critères	Critères vérifiables avant mise en production

1

RÉSUMÉ EXÉCUTIF

Vision produit · métriques clés · alignement plateforme

Ce document est la spécification technique complète du Module Finance & Gestion de Leopardo RH. Il permet à un développeur senior de concevoir, implémenter et tester le module de façon autonome. Tout est spécifié : architecture, base de données, API, écrans mobiles, rôles, tâches.

Aspect	Détail
Problème client	Les PME gèrent leurs factures sur Excel ou dans un logiciel séparé. Aucun lien avec leur gestion RH. Tout est en silos.
Solution	Module intégré dans Leopardo RH : factures, dépenses, achats, trésorerie — accessible depuis l'app mobile du manager et du gestionnaire financier.
Rôle clé	Gestionnaire financier — nouveau sous-rôle manager avec accès complet au module finance et zéro accès aux données RH.
Surfaces	App Flutter (mobile-first) : 18 écrans. Dashboard Blade (web) : gestion avancée et exports.
Plans cibles	Business : factures + dépenses. Entreprise : + achats + trésorerie + analytiques + intégration bancaire.
Effort total	~42 jours-développeur · 65 tâches · 7 semaines · 6 sprints.
Dépendance	MVP Phase 1 core et design system doivent être stables et déployés.
Couleur domaine	Ambre (#D97706) — couleur officielle du domaine Finance dans le design system Leopardo RH.
PDF finances	Factures et rapports générés par DomPDF (déjà dans le projet) — réutilisation du service existant.

2

L'OPPORTUNITÉ & POSITIONNEMENT

Marché, valeur, nouveau rôle, modèle économique

2.1 — Contexte et problème client

La grande majorité des PME ciblées par Leopardo RH (5-50 employés, Maghreb + Méditerranée) gèrent leurs finances dans trois endroits différents : Excel pour les factures, un registre papier pour les dépenses courantes, et un logiciel de comptabilité déconnecté pour les achats. Le résultat est une perte de temps considérable et une impossibilité de voir la santé financière globale en temps réel.

Le manager d'une PME a besoin de répondre à 4 questions simples en moins de 30 secondes depuis son téléphone :

1. Combien me doivent mes clients ? (factures en attente)
2. Combien ai-je dépensé ce mois ? (dépenses réelles vs budget)
3. Quelles commandes d'achat sont en cours ? (achats fournisseurs)
4. Quel est mon solde de trésorerie actuel ? (cash disponible)

Aucun concurrent sur les marchés MENA et Méditerranée ne répond à ces 4 questions depuis une seule app mobile qui inclut aussi la gestion RH. C'est le gap exact que le module Finance de Leopardo RH adresse.

2.2 — Proposition de valeur

Pour le manager : voir en un coup d'œil sur son mobile l'état financier de son entreprise, sans ouvrir Excel ni appeler son comptable. Le module finance s'ouvre depuis le même BottomNav que le module RH — même app, même login, même expérience.

Pour le gestionnaire financier : faire tout son travail (saisie, validation, envoi de factures, enregistrement de dépenses) depuis l'app mobile sur le terrain, sans avoir besoin d'accéder à un PC. Il voit uniquement ce qui le concerne — aucune donnée RH, aucun salaire d'employé.

Pour le comptable/expert tiers (Phase 2) : exporter les données structurées (JSON, CSV, ou format compatible SAGE/ERP) pour la liasse fiscale sans ressaisie manuelle.

2.3 — Le rôle Gestionnaire financier — cœur du module

C'est la décision de conception la plus importante : le gestionnaire financier est un sous-rôle manager (role='manager', manager_role='finance') avec accès TOTAL au module finance et accès ZÉRO aux données RH. Il ne voit ni les salaires, ni les pointages, ni les absences des employés. Cette séparation est fondamentale pour la confiance.

Comparaison des accès :

Ce que voit/fait...	Manager Principal	Gestionnaire Finance	Employé
Tableau de bord finance	✓	✓	—
Créer une facture	✓	✓	—
Envoyer une facture par email	✓	✓	—
Valider une dépense	✓	✓	—
Saisir une dépense	✓	✓	—
Créer un bon de commande	✓	✓	—
Voir les soldes de trésorerie	✓	✓	—
Voir les salaires des employés	✓	—	—
Voir les pointages	✓	—	—
Accéder aux données RH	✓	—	—
Gérer les autres employés	✓	—	—
Configurer les budgets	✓	—	—
Exporter données financières	✓	✓	—

2.4 — Impact sur le modèle économique

Le module Finance justifie une augmentation substantielle de la valeur perçue des plans Business et Enterprise. Il transforme Leopard RH d'outil RH en plateforme de gestion opérationnelle PME — ce qui légitime une tarification plus élevée à terme.

Plan	Fonctions Finance incluses	Limite	Impact
Trial	Aucune	—	Incitation upgrade Business
Starter	Aucune	—	Argument de vente fort vers Business
Business	Factures + Dépenses	100 doc./mois	Rétention + upsell Enterprise
Enterprise	Tout (Factures + Dépenses + Achats + Trésorerie + Analytiques)	Illimité	Rétention forte + nouvelle cible PME 20-100 employés

2.5 — Analyse risques

Risque	Probabilité	Mitigation
Résistance culturelle à la saisie mobile	Haute dans certains marchés	UX ultra-simplifiée. 3 taps max pour saisir une dépense. Mode vocal en Phase 3.
Confusion avec logiciel comptable	Moyenne	Positionner clairement : gestion opérationnelle, pas comptabilité fiscale. Export vers comptable = Phase 2.
Erreurs de saisie (montants, devises)	Haute	Validation stricte côté API. Confirmation avant soumission. Correction possible dans les 24h.
Volume important de pièces jointes (photos reçus)	Haute	Compression image côté client. Stockage S3 avec purge après 90j (configurable par plan).
Double saisie (mobile + web) créant des doublons	Faible	Verrouillage optimiste (version timestamp). Conflit affiché clairement avec résolution manuelle.
Données financières exposées en cas de fuite	Critique	Chiffrement au repos. Audit log de tous les accès. Isolation tenant stricte. Pas de données financières dans les logs.

3

ARCHITECTURE TECHNIQUE

Stack · flux · composants · sécurité · PDF

3.1 — Vue d'ensemble du système

Principe de conception : le module Finance réutilise 100% de l'infrastructure existante de Leopard RH. Pas de nouvelle base de données, pas de nouveau service externe obligatoire en Phase 1. 9 nouvelles tables dans le schéma tenant existant, des endpoints API V1 supplémentaires, et de nouveaux écrans Flutter. C'est tout.

MODULE FINANCE & GESTION

App Flutter (mobile-first)

Dashboard Web (Blade)

FinanceDashboardScreen

/finance/dashboard

InvoiceListScreen

/finance/invoices

InvoiceCreateScreen

/finance/expenses

ExpenseListScreen

/finance/purchases

ExpenseCreateScreen

/finance/treasury

PurchaseScreen

/finance/reports

TreasuryScreen

API REST

Laravel 11 API V1 (existant)

/api/v1/finance/invoices

/api/v1/finance/expenses

/api/v1/finance/purchases

/api/v1/finance/treasury

/api/v1/finance/reports

/api/v1/finance/suppliers

PostgreSQL 16 (Neon.tech existant) – schéma tenant

invoices · invoice_items · expenses · expense_categories

purchase_orders · purchase_items · suppliers

finance_accounts · finance_transactions

Services partagés : DomPDF (PDF) · S3/local (pièces jointes)

Queue Laravel (envoi email asynchrone) · Sanctum (auth existant)

3.2 — Flux de données financières

Trois flux principaux couvrent 90% des cas d'usage :

Flux 1 — Création et envoi d'une facture client :

```
Gestionnaire (Flutter) → POST /finance/invoices
→ Laravel valide (rôle finance + plan) → crée invoice + items en transaction DB
→ retourne invoice avec id et statut 'draft'
Gestionnaire → POST /finance/invoices/{id}/send
→ Laravel génère PDF via DomPDF (template invoice.blade.php)
→ Dispatch job SendInvoiceEmailJob(invoice, pdf_path)
→ Job envoie email avec PDF en pièce jointe → met à jour statut 'sent'
→ Notification push Flutter au manager (via Firebase ou APNs)
```

Flux 2 — Saisie d'une dépense avec photo du reçu :

```
Gestionnaire (Flutter) → capture photo reçu → compression JPEG 60%
→ POST /finance/expenses/upload-receipt (multipart/form-data)
→ Laravel valide taille (max 5MB) → stocke dans storage/receipts/{company}/{date}/
→ retourne { receipt_path, receipt_url }
Gestionnaire → POST /finance/expenses
→ body : { amount, category_id, description, date, receipt_path, payment_method }
→ Laravel crée expense → si > seuil_validation du plan → statut 'pending_approval'
→ si < seuil → statut 'approved' automatique → mise à jour balance du compte
```

Flux 3 — Tableau de bord trésorerie temps réel :

```
App Flutter (TimerProvider, refresh toutes les 5min)
→ GET /finance/dashboard → Laravel calcule agrégats depuis finance_transactions
→ { balance_total, invoices_pending, expenses_month, cash_flow_7d[] }
→ Flutter affiche 4 MetricCards + LineChart (fl_chart) + liste transactions récentes
→ Pas de cache côté client : toujours données fraîches depuis la DB
```

3.3 — Sécurité et isolation des données financières

Les données financières sont parmi les plus sensibles de l'application. Règles absolues :

- **Isolation tenant stricte** : toutes les tables finance ont un company_id avec Global Scope Eloquent. Impossible d'accéder aux données d'une autre entreprise par construction.
- **Double vérification des permissions** : middleware chain = [sanctum, tenant, check_subscription:finance, finance_role]. Chaque couche est indépendante.
- **Chiffrement des données sensibles** : les IBAN, numéros de compte bancaire et informations de carte sont chiffrés en base via Crypt::encryptString(). Jamais en clair dans les logs.
- **Audit log obligatoire** : chaque action sur une facture (création, modification, envoi, suppression) est enregistrée dans finance_audit_log avec user_id, timestamp, IP, et diff des changements.
- **Montants en entiers (cents)** : tous les montants sont stockés en centimes (integer) dans PostgreSQL. Jamais de DECIMAL pour les montants financiers — évite les erreurs d'arrondi flottant.

Convention montants : amount = 125000 signifie 1 250,00 dans la devise de la company. La conversion display se fait dans les API Resources Laravel et dans les widgets Flutter (FinanceFormatter.format(amount, currency)). Cette convention est identique à celle de Stripe.

3.4 — Génération PDF (factures & rapports)

DomPDF est déjà dans le projet (utilisé pour les reçus de pointage). Le module finance réutilise ce service avec de nouveaux templates Blade.

```
// app/Services/FinancePdfService.php
class FinancePdfService {
    public function generateInvoice(Invoice $invoice): string {
        $invoice->load(['items', 'client', 'company', 'createdBy']);
        $pdf = Pdf::loadView('finance.pdf.invoice', compact('invoice'))
            ->setPaper('a4')->setOption('dpi', 150);
        $path = "invoices/{$invoice->company_id}/{$invoice->number}.pdf";
        Storage::put($path, $pdf->output());
        $invoice->update(['pdf_path' => $path]);
        return $path;
    }

    public function generateMonthlyReport(Company $co, int $year, int $month): string {
        // agrège invoices, expenses, transactions du mois
        $data = $this->aggregateMonth($co, $year, $month);
        $pdf = Pdf::loadView('finance.pdf.monthly-report', $data)->setPaper('a4');
        $path = "reports/{$co->id}/{$year}-{$month:02d}-report.pdf";
        Storage::put($path, $pdf->output());
        return $path;
    }
}
```

4

MODÈLE DE DONNÉES COMPLET

9 nouvelles tables dans le schéma tenant + modifications existantes

4.1 — Table : invoices (factures clients)

Table centrale du module. Une facture peut passer par plusieurs statuts et peut être liée à une ou plusieurs commandes d'achat en Phase 2.

Colonne	Type	Contraintes	Description
id	INT	PK AUTO	
company_id	UUID	FK · NOT NULL · INDEX	Isolation tenant — Global Scope
number	VARCHAR(30)	NOT NULL · UNIQUE par company	Numéro séquentiel : INV-2026-0001
client_name	VARCHAR(150)	NOT NULL	Nom du client (dénormalisé pour historique)
client_email	VARCHAR(150)	NULL	Email pour envoi automatique
client_address	TEXT	NULL	Adresse complète
client_tax_id	VARCHAR(50)	NULL	NIF/RC/SIRET selon le pays
status	VARCHAR(20)	NOT NULL DEFAULT 'draft'	draft sent paid partial overdue cancelled
issue_date	DATE	NOT NULL DEFAULT NOW()	Date d'émission
due_date	DATE	NOT NULL	Date d'échéance (calculée selon payment_terms)
payment_terms	SMALLINT	NOT NULL DEFAULT 30	Délai de paiement en jours
subtotal_cents	BIGINT	NOT NULL DEFAULT 0	Sous-total HT en centimes
tax_rate	DECIMAL(5,2)	NOT NULL DEFAULT 0	Taux TVA (ex: 19.00 pour 19%)
tax_cents	BIGINT	NOT NULL DEFAULT 0	Montant TVA en centimes (calculé)
total_cents	BIGINT	NOT NULL DEFAULT 0	Total TTC en centimes
paid_cents	BIGINT	NOT NULL DEFAULT 0	Montant déjà encaissé
currency	CHAR(3)	NOT NULL DEFAULT 'DZD'	ISO 4217 : DZD, MAD, TND, EUR, TRY
notes	TEXT	NULL	Notes internes (non affichées sur le PDF)
footer_text	TEXT	NULL	Texte pied de page du PDF
pdf_path	VARCHAR(255)	NULL	Chemin stockage PDF généré
sent_at	TIMESTAMPTZ	NULL	Date d'envoi par email

Colonne	Type	Contraintes	Description
paid_at	TIMESTAMPTZ	NULL	Date de paiement complet
created_by	INT	FK → employees.id	
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	
deleted_at	TIMESTAMPTZ	NULL	Soft delete — SoftDeletes

4.2 — Table : invoice_items (lignes de facture)

Colonne	Type	Contraintes	Description
id	INT	PK AUTO	
invoice_id	INT	FK → invoices.id · CASCADE	
company_id	UUID	FK · NOT NULL	Dénormalisé pour Global Scope
description	VARCHAR(255)	NOT NULL	Description de la prestation/produit
quantity	DECIMAL(10,3)	NOT NULL DEFAULT 1	Quantité (supporte fractions : 2.5h)
unit	VARCHAR(30)	NULL	heure, jour, unité, kg, forfait...
unit_price_cents	BIGINT	NOT NULL	Prix unitaire HT en centimes
total_cents	BIGINT	NOT NULL	quantity × unit_price_cents (calculé)
sort_order	SMALLINT	NOT NULL DEFAULT 0	Ordre d'affichage sur le PDF

4.3 — Table : expenses (dépenses)

Colonne	Type	Contraintes	Description
id	INT	PK AUTO	
company_id	UUID	FK · NOT NULL · INDEX	
category_id	INT	FK → expense_categories.id	NULL si catégorie supprimée
title	VARCHAR(200)	NOT NULL	Libellé de la dépense
description	TEXT	NULL	Détail optionnel
amount_cents	BIGINT	NOT NULL	Montant en centimes
currency	CHAR(3)	NOT NULL DEFAULT 'DZD'	
expense_date	DATE	NOT NULL	Date réelle de la dépense
payment_method	VARCHAR(30)	NOT NULL DEFAULT 'cash'	cash card transfer check
status	VARCHAR(20)	NOT NULL DEFAULT 'approved'	draft pending_approval approved rejected
receipt_path	VARCHAR(255)	NULL	Photo du reçu (S3 ou stockage local)
receipt_url	VARCHAR(500)	NULL	URL publique temporaire (générée à la demande)
supplier_id	INT	FK → suppliers.id · NULL	Fournisseur associé si connu
account_id	INT	FK → finance_accounts.id · NULL	Compte débité (trésorerie)
approved_by	INT	FK → employees.id · NULL	Manager ayant approuvé
approved_at	TIMESTAMPZ	NULL	
rejection_reason	TEXT	NULL	Motif de rejet
created_by	INT	FK → employees.id · NOT NULL	
created_at	TIMESTAMPZ	NOT NULL	
updated_at	TIMESTAMPZ	NOT NULL	

4.4 — Table : expense_categories

Colonne	Type	Description
id	INT PK	
company_id	UUID FK	Catégories personnalisables par entreprise

Colonne	Type	Description
name	VARCHAR(100) NOT NULL	Transport, Fournitures, Repas, IT, Marketing...
icon	VARCHAR(50) NULL	Nom icône MaterialIcons ou Heroicon
color	VARCHAR(7) NULL	Hex color pour affichage dans l'app (#F59E0B)
budget_monthly_cents	BIGINT NULL	Budget mensuel optionnel pour alertes
is_active	BOOLEAN NOT NULL DEFAULT true	
sort_order	SMALLINT NOT NULL DEFAULT 0	
created_at	TIMESTAMPTZ NOT NULL	

4.5 — Table : purchase_orders (bons de commande / achats fournisseurs)

Colonne	Type	Contraintes	Description
id	INT	PK AUTO	
company_id	UUID	FK · NOT NULL	
number	VARCHAR(30)	NOT NULL · UNIQUE par company	PO-2026-0001
supplier_id	INT	FK → suppliers.id · NOT NULL	
status	VARCHAR(20)	NOT NULL DEFAULT 'draft'	draft sent confirmed received invoiced cancelled
order_date	DATE	NOT NULL DEFAULT NOW()	
expected_delivery	DATE	NULL	Date de livraison attendue
delivery_date	DATE	NULL	Date réelle de réception
subtotal_cents	BIGINT	NOT NULL DEFAULT 0	
tax_cents	BIGINT	NOT NULL DEFAULT 0	
total_cents	BIGINT	NOT NULL DEFAULT 0	
currency	CHAR(3)	NOT NULL DEFAULT 'DZD'	
notes	TEXT	NULL	
pdf_path	VARCHAR(255)	NULL	PDF du bon de commande
linked_expense_id	INT	FK → expenses.id · NULL	Si la commande génère une dépense

Colonne	Type	Contraintes	Description
created_by	INT	FK → employees.id	
created_at	TIMESTAMPTZ	NOT NULL	
updated_at	TIMESTAMPTZ	NOT NULL	

4.6 — Table : purchase_items

Colonne	Type	Description
id	INT PK	
purchase_order_id	INT FK → purchase_orders.id CASCADE	
company_id	UUID FK	
description	VARCHAR(255) NOT NULL	Désignation du produit/service acheté
quantity	DECIMAL(10,3) NOT NULL DEFAULT 1	
unit	VARCHAR(30) NULL	
unit_price_cents	BIGINT NOT NULL	Prix unitaire HT en centimes
total_cents	BIGINT NOT NULL	quantity × unit_price_cents
received_quantity	DECIMAL(10,3) NOT NULL DEFAULT 0	Qté réellement reçue (pour réception partielle)
sort_order	SMALLINT NOT NULL DEFAULT 0	

4.7 — Table : suppliers (fournisseurs)

Colonne	Type	Description
id	INT PK	
company_id	UUID FK · NOT NULL	
name	VARCHAR(150) NOT NULL	Nom commercial du fournisseur
contact_name	VARCHAR(100) NULL	Nom du contact principal
email	VARCHAR(150) NULL	
phone	VARCHAR(30) NULL	
address	TEXT NULL	
tax_id	VARCHAR(50) NULL	NIF, RC, SIRET selon le pays
payment_terms	SMALLINT NOT NULL DEFAULT 30	Délai de paiement habituel (jours)
iban	TEXT NULL	Chiffré AES-256 via Crypt::encryptString()

Colonne	Type	Description
notes	TEXT NULL	
is_active	BOOLEAN NOT NULL DEFAULT true	
created_at	TIMESTAMPTZ NOT NULL	
updated_at	TIMESTAMPTZ NOT NULL	

4.8 — Table : finance_accounts (comptes de trésorerie)

Représente les comptes bancaires, caisses, et portefeuilles de l'entreprise. Chaque transaction est rattachée à un compte.

Colonne	Type	Description
id	INT PK	
company_id	UUID FK · NOT NULL	
name	VARCHAR(100) NOT NULL	'Compte CPA principal', 'Caisse magasin', 'Caisse chantier'
type	VARCHAR(20) NOT NULL	bank cash mobile_money other
currency	CHAR(3) NOT NULL	ISO 4217
initial_balance_cents	BIGINT NOT NULL DEFAULT 0	Solde initial lors de la création du compte
current_balance_cents	BIGINT NOT NULL DEFAULT 0	Solde actuel (mis à jour par trigger ou job)
bank_name	VARCHAR(100) NULL	Nom de la banque
account_number	TEXT NULL	Chiffré AES-256
iban	TEXT NULL	Chiffré AES-256
is_default	BOOLEAN NOT NULL DEFAULT false	Compte par défaut pour les nouvelles opérations
is_active	BOOLEAN NOT NULL DEFAULT true	
created_at	TIMESTAMPTZ NOT NULL	
updated_at	TIMESTAMPTZ NOT NULL	

4.9 — Table : `finance_transactions` (livre de comptes)

Enregistre chaque mouvement de trésorerie. La balance des comptes est calculée depuis cette table — c'est la source de vérité.

Colonne	Type	Contraintes	Description
<code>id</code>	BIGINT	PK AUTO	ID grand comme le nombre de transactions
<code>company_id</code>	UUID	FK · NOT NULL · INDEX	
<code>account_id</code>	INT	FK → <code>finance_accounts.id</code> · NOT NULL	
<code>type</code>	VARCHAR(20)	NOT NULL	income expense transfer_in transfer_out
<code>amount_cents</code>	BIGINT	NOT NULL	Toujours positif — le type indique le sens
<code>balance_after_cents</code>	BIGINT	NOT NULL	Solde du compte après cette transaction
<code>currency</code>	CHAR(3)	NOT NULL	
<code>description</code>	VARCHAR(255)	NOT NULL	Libellé de la transaction
<code>reference_type</code>	VARCHAR(50)	NULL	'invoice' 'expense' 'purchase_order' 'manual'
<code>reference_id</code>	INT	NULL	ID de l'entité liée (polymorphe)
<code>transaction_date</code>	DATE	NOT NULL	Date comptable
<code>created_by</code>	INT	FK → <code>employees.id</code>	
<code>created_at</code>	TIMESTAMPTZ	NOT NULL	

Important : `balance_after_cents` est dénormalisé (calculé et stocké). Cette redondance est intentionnelle — elle permet d'afficher l'historique de solde sans recalcul. La cohérence est garantie par une DB transaction : toujours créer la `finance_transaction` ET mettre à jour `finance_accounts.current_balance_cents` dans la même transaction PostgreSQL.

4.10 — Modifications tables existantes

Table	Modification	SQL
<code>plans</code>	Ajouter dans <code>features JSONB</code> : <code>finance_enabled</code> , <code>finance_max_docs</code> , <code>finance_modules[]</code>	<code>UPDATE plans SET features = features '{"finance_enabled":true,"finance_max_docs":100,"finance_modules":["invoices","expenses"]}' WHERE name='Business'</code>
<code>companies</code>	Aucune — <code>company_id</code> suffit	—
<code>employees</code>	Aucune — relations via RBAC existant	—

Index critiques

```
CREATE INDEX idx_invoices_company_status ON invoices(company_id, status) WHERE deleted_at IS NULL;
CREATE INDEX idx_invoices_due_date ON invoices(company_id, due_date) WHERE status NOT IN ('paid', 'cancelled');
CREATE INDEX idx_expenses_company_date ON expenses(company_id, expense_date DESC);
CREATE INDEX idx_expenses_category ON expenses(category_id, company_id);
CREATE INDEX idx_transactions_account ON finance_transactions(account_id, transaction_date DESC);
CREATE INDEX idx_transactions_company ON finance_transactions(company_id, transaction_date DESC);
CREATE INDEX idx_po_supplier ON purchase_orders(supplier_id, company_id) WHERE deleted_at IS NULL;
```

5

RBAC — MATRICE COMPLÈTE DES PERMISSIONS

Manager Principal · Gestionnaire Financier · Autres rôles

*Le Gestionnaire Financier (manager_role='finance') a accès TOTAL au module finance et ZÉRO accès aux données RH.
Cette coupure est implémentée dans FinancePolicy::class et dans le middleware CheckFinanceRole.*

Action	Super Admin	Manager Principal	Gestionnaire Finance	RH / Dept	Employé
Voir dashboard finance global	✓	✓	✓	✗	✗
Voir KPIs et graphiques	✓	✓	✓	✗	✗
Lister toutes les factures	✓	✓	✓	✗	✗
Créer une facture	✓	✓	✓	✗	✗
Modifier une facture (draft)	✓	✓	✓	✗	✗
Envoyer une facture par email	✓	✓	✓	✗	✗
Marquer facture comme payée	✓	✓	✓	✗	✗
Supprimer une facture	✓	✓	✗	✗	✗
Télécharger le PDF d'une facture	✓	✓	✓	✗	✗
Lister toutes les dépenses	✓	✓	✓	✗	✗
Saisir une dépense	✓	✓	✓	✗	✗
Uploader une photo de reçu	✓	✓	✓	✗	✗
Approuver une dépense	✓	✓	✗	✗	✗
Rejeter une dépense	✓	✓	✗	✗	✗
Configurer les catégories	✓	✓	✗	✗	✗
Créer un bon de commande	✓	✓	✓	✗	✗
Envoyer BC au fournisseur	✓	✓	✓	✗	✗
Confirmer réception	✓	✓	✓	✗	✗
Gérer les fournisseurs	✓	✓	✓	✗	✗
Voir soldes des comptes	✓	✓	✓	✗	✗
Créer un compte de trésorerie	✓	✓	✗	✗	✗
Saisir une transaction manuelle	✓	✓	✓	✗	✗
Générer rapport mensuel PDF	✓	✓	✓	✗	✗

Action	Super Admin	Manager Principal	Gestionnaire Finance	RH / Dept	Employé
Exporter données CSV/JSON	✓	✓	✓	✗	✗

6

ÉCRANS MOBILE — SPÉCIFICATIONS COMPLÈTES

18 écrans Flutter détaillés · navigation · composants

6.1 — Structure de navigation (BottomNav adaptative)

Le module Finance s'intègre dans la navigation adaptative définie dans le Design System. Un onglet 'Finance' (icône `account_balance`, couleur amber) apparaît dans le `BottomNavigationBar` uniquement si `plan.features.finance_enabled = true`.

```
// lib/core/navigation/module_registry.dart – ajout du module finance
AppTab(
  id: 'finance',
  label: 'Finance',
  icon: Icons.account_balance_outlined,
  activeIcon: Icons.account_balance,
  moduleId: 'finance',
  domain: Domain.finance, // → couleur ambre partout dans ce tab
  route: '/finance',
),

// Routes GoRouter – lib/core/navigation/app_router.dart
GoRoute(path: '/finance', builder: (_, __) => FinanceDashboardScreen()),
GoRoute(path: '/finance/invoices', builder: (_, __) => InvoiceListScreen()),
GoRoute(path: '/finance/invoices/new', builder: (_, __) => InvoiceCreateScreen()),
GoRoute(path: '/finance/invoices/:id', builder: (_, s) => InvoiceDetailScreen(id: s.pathParameters['id']!)),
GoRoute(path: '/finance/expenses', builder: (_, __) => ExpenseListScreen()),
GoRoute(path: '/finance/expenses/new', builder: (_, __) => ExpenseCreateScreen()),
GoRoute(path: '/finance/purchases', builder: (_, __) => PurchaseListScreen()),
GoRoute(path: '/finance/treasury', builder: (_, __) => TreasuryScreen()),
GoRoute(path: '/finance/suppliers', builder: (_, __) => SupplierListScreen()),
```

6.2 — Les 18 écrans — spécifications

SCR-F01

Finance Dashboard

Les deux

Écran d'accueil du module. 4 MetricCards horizontalement défilables : Factures en attente (total en DZD + nb), Dépenses du mois, Solde trésorerie, Achats en cours. LineChart 7 jours (`fl_chart`) des entrées/sorties. Liste des 5 dernières transactions avec icône type + montant coloré. FAB contextuel selon rôle : Manager → 'Nouvelle facture', Gestionnaire → même.

Composants : FinanceMetricCard · FinanceCashFlowChart · RecentTransactionTile · DomainFAB

SCR-F02

Liste Factures

Les deux

ListView paginée des factures. Tri par date / échéance / montant. Filtres : statut (chips : Brouillon, Envoyée, Payée, En retard). Chaque tile : numéro, nom client, montant, badge statut ambre/vert/rouge. Swipe-to-action : Envoyer / Marquer payée. Pull-to-refresh. SearchBar par client ou numéro.

Composants : InvoiceTile · StatusBadge(domain:finance) · SwipeAction · SearchBar

SCR-F03

Créer / Modifier Facture

Les deux

Formulaire multi-étapes : Étape 1 — Infos client (nom, email, adresse, NIF). Étape 2 — Lignes de facture : liste de InvoiceItemRow avec description + quantité + prix unitaire + total calculé. Bouton + Ajouter ligne. Étape 3 — Conditions : date émission, échéance, taux TVA (sélecteur par pays), devise. Notes et texte pied de page. Récapitulatif HT / TVA / TTC. Bouton Enregistrer brouillon + Enregistrer et envoyer.

Composants : InvoiceItemRow · TaxRateSelector · CurrencyInput · AmountDisplay

SCR-F04

Détail Facture

Les deux

Affichage complet PDF-like de la facture : en-tête entreprise + client, tableau lignes, totaux. Timeline des événements : Créée le X par Y, Envoyée le X, Payée le X. Boutons d'action contextuels selon statut : [Envoyer] si draft, [Marquer payée] si sent, [Télécharger PDF] toujours. Historique des modifications.

Composants : InvoicePreviewWidget · TimelineEvent · ActionButtonGroup

SCR-F05

Paiement reçu (enregistrer)

Les deux

BottomSheet ou écran dédié pour enregistrer un paiement : montant reçu, date, mode de paiement (virement, espèces, chèque, CB), compte crédité (sélection finance_accounts), référence optionnelle. Si montant = total → statut 'paid'. Si partiel → statut 'partial' avec solde restant affiché.

Composants : PaymentMethodPicker · AccountSelector · AmountInput

SCR-F06

Liste Dépenses

Les deux

ListView groupée par mois. Barre de progression mensuelle (dépenses réelles / budget si configuré). Filtres : catégorie (chips colorés) + statut + période. Chaque tile : icône catégorie coloré + titre + montant + date + badge statut. Swipe pour voir le reçu photo. Totaux mensuels en haut.

Composants : ExpenseTile · CategoryChip · MonthlyProgressBar · ReceiptPreview

SCR-F07

Saisir une Dépense

Les deux

Formulaire 1 page (pas de multi-étapes — doit être ultra-rapide). Champs dans l'ordre d'importance : Montant (grand, premier champ, clavier numérique), Catégorie (sélecteur avec icônes et couleurs), Titre (texte), Date (DatePicker, défaut aujourd'hui), Mode de paiement (chips), Description (optionnel), Photo reçu (CameraButton). Bouton Enregistrer en bas. Max 3 taps pour une dépense courante.

Composants : BigAmountInput · CategoryPicker · CameraReceiptButton · QuickDateChips

SCR-F08

Photo reçu plein écran

Les deux

Visualiseur photo du reçu avec possibilité de zoom (InteractiveViewer), rotation, et partage. Bouton pour remplacer la photo. Affiche les métadonnées : date, montant de la dépense liée, catégorie.

Composants : InteractiveViewer · ReceiptMetaBar · ReplacePhotoButton

SCR-F09**Liste Achats / Bons de commande**

Les deux

ListView des bons de commande groupés par statut. Timeline verticale montrant l'avancement : Brouillon → Envoyé → Confirmé → Reçu → Facturé. Chaque tile : numéro BC, nom fournisseur, montant total, statut avec couleur. Bouton de changement de statut inline.

Composants : PurchaseOrderTile · StatusStepper · SupplierAvatar

SCR-F10**Créer Bon de Commande**

Les deux

Formulaire similaire à la facture : sélection fournisseur (avec recherche dans la liste), lignes de commande, date livraison attendue, notes. Bouton Envoyer par email au fournisseur (génère PDF BC). Possibilité de lier à une dépense existante.

Composants : SupplierSearchSelect · PurchaseItemRow · DeliveryDatePicker

SCR-F11**Liste Fournisseurs**

Les deux

Annuaire des fournisseurs. Chaque tile : nom, contact, nb commandes, délai habituel. Swipe to call/email. Tap → Détail fournisseur avec historique des commandes.

Composants : SupplierTile · ContactActionChips · HistoryTimeline

SCR-F12**Trésorerie — Soldes**

Manager

Vue des comptes de trésorerie. Chaque compte : nom, type (banque/caisse), solde actuel coloré (vert si positif, rouge si négatif), devise. Graphique évolution sur 30 jours par compte. Bouton + Transaction manuelle. Total consolidé tous comptes en haut.

Composants : AccountCard · BalanceChart · ConsolidatedBalance

SCR-F13**Transactions d'un compte**

Manager

Liste chronologique des transactions d'un compte. Chaque ligne : date + description + montant signé (+ en vert, - en rouge) + solde après. Filtre par type. Export CSV depuis ce screen.

Composants : TransactionTile · SignedAmountDisplay · ExportButton

SCR-F14**Saisir Transaction manuelle**

Les deux

Formulaire : type (entrée/sortie), montant, description, date, compte, référence optionnelle. Confirmation obligatoire avant validation (résumé avant soumission).

Composants : TransactionTypeToggle · AccountSelector · ConfirmationDialog

SCR-F15**Rapports mensuels**

Les deux

Sélecteur mois/année. Résumé : factures émises, encaissées, dépenses, solde net. Bouton Générer PDF (job asynchrone → notification push quand prêt). Bouton Exporter CSV. Graphiques : revenus vs dépenses barre, répartition dépenses par catégorie camembert.

Composants : MonthYearPicker · FinanceSummaryCard · BarChart · PieChart · AsyncPdfButton

SCR-F16 **Tableau de bord Gestionnaire**

Gestionnaire

Version simplifiée du dashboard orientée tâches : 'À traiter aujourd'hui' (dépenses en attente validation, factures à envoyer, bons de commande à confirmer). Liste de tâches prioritaires avec compteurs. Bouton d'action direct depuis chaque item.

Composants : TaskCountCard · PriorityTaskList · QuickActionButton

SCR-F17 **Notifications Finance**

Les deux

Ecran dédié aux notifications financières : factures en retard (overdue), dépenses en attente d'approbation, livraisons attendues aujourd'hui. Tap sur chaque notification → deep link vers l'entité concernée. Badge rouge sur l'onglet Finance du BottomNav.

Composants : FinanceNotifTile · DeepLinkHandler · BadgeCount

SCR-F18 **Paramètres Finance**

Manager

Configuration du module : numérotation factures (préfixe, numéro suivant), taux TVA par défaut, délai de paiement par défaut, seuil d'approbation des dépenses (montant au-dessus duquel une dépense doit être approuvée par le manager), comptes de trésorerie (ajouter/modifier/supprimer), gestion des catégories de dépenses.

Composants : SettingsSection · TaxRateConfig · ApprovalThresholdSlider · AccountManager

7

CONTRATS API COMPLETS

Tous les endpoints · schemas JSON · codes d'erreur

7.1 — Endpoints Factures (/api/v1/finance/invoices)

Méthode	Endpoint	Rôle min.	Description
GET	/finance/invoices	finance	Liste paginée + filtres (status, client, date_from, date_to)
POST	/finance/invoices	finance	Crée facture + items en transaction DB. Calcule totaux.
GET	/finance/invoices/{id}	finance	Détail complet + items + historique événements
PUT	/finance/invoices/{id}	finance	Modifie (seulement si statut=draft)
DELETE	/finance/invoices/{id}	manager:principal	Soft delete (seulement si draft)
POST	/finance/invoices/{id}/send	finance	Génère PDF + envoi email + statut=sent
POST	/finance/invoices/{id}/payment	finance	Enregistre paiement (partiel ou total)
GET	/finance/invoices/{id}/pdf	finance	Download PDF (génère si inexistant)
POST	/finance/invoices/{id}/duplicate	finance	Crée brouillon copie (client + items)
GET	/finance/invoices/overdue	finance	Liste factures en retard (due_date < today, non payées)

7.2 — Endpoints Dépenses (/api/v1/finance/expenses)

Méthode	Endpoint	Rôle min.	Description
GET	/finance/expenses	finance	Liste + filtres (category_id, status, date_from, date_to)
POST	/finance/expenses	finance	Crée dépense. Si amount > seuil → statut pending_approval.
GET	/finance/expenses/{id}	finance	Détail + URL reçu présignée (TTL=15min)
PUT	/finance/expenses/{id}	finance	Modifie (si statut=draft ou approved par le créateur)
DELETE	/finance/expenses/{id}	finance	Soft delete
POST	/finance/expenses/upload-receipt	finance	Upload photo reçu → retourne {receipt_path, receipt_url}

Méthode	Endpoint	Rôle min.	Description
POST	/finance/expenses/{id}/approve	manager:principal	Approuve. Crée finance_transaction si account_id fourni.
POST	/finance/expenses/{id}/reject	manager:principal	Rejette avec rejection_reason obligatoire
GET	/finance/expenses/categories	finance	Liste des catégories actives
POST	/finance/expenses/categories	manager:principal	Crée une catégorie
GET	/finance/expenses/summary	finance	Totaux par catégorie pour le mois courant

7.3 — Endpoints Achats & Fournisseurs

Méthode	Endpoint	Description
GET	/finance/purchase-orders	Liste BC + filtres (status, supplier_id, date)
POST	/finance/purchase-orders	Crée BC + items. Génère numéro PO-YYYY-NNNN.
PUT	/finance/purchase-orders/{id}	Modifie (si draft)
POST	/finance/purchase-orders/{id}/send	Génère PDF + email au fournisseur
POST	/finance/purchase-orders/{id}/confirm	Fournisseur a confirmé → statut confirmed
POST	/finance/purchase-orders/{id}/receive	Réception marchandises (quantités reçues par item)
GET	/finance/suppliers	Liste fournisseurs actifs
POST	/finance/suppliers	Crée fournisseur
PUT	/finance/suppliers/{id}	Modifie
GET	/finance/suppliers/{id}/orders	Historique commandes d'un fournisseur

7.4 — Endpoints Trésorerie & Rapports

Méthode	Endpoint	Description
GET	/finance/dashboard	KPIs globaux + cash flow 7j + dernières transactions
GET	/finance/accounts	Liste des comptes de trésorerie avec soldes
POST	/finance/accounts	Crée un compte
GET	/finance/accounts/{id}/transactions	Transactions d'un compte (paginées, filtres date)
POST	/finance/transactions	Saisie manuelle d'une transaction
GET	/finance/reports/monthly	Rapport mensuel : agrégats + breakdown par catégorie
POST	/finance/reports/monthly/pdf	Génère PDF rapport mois (job async → webhook ou polling)

Méthode	Endpoint	Description
GET	/finance/reports/export	Export CSV/JSON des données financières (période à définir)

7.5 — Schema JSON : réponse GET /finance/dashboard

```
{
  "period": { "start": "2026-04-01", "end": "2026-04-30" },
  "kpis": {
    "invoices_pending_cents": 345000,    // 3 450,00 DZD
    "invoices_pending_count": 3,
    "invoices_overdue_cents": 120000,
    "invoices_overdue_count": 1,
    "expenses_month_cents": 87500,      // dépenses approuvées ce mois
    "balance_total_cents": 1250000,     // somme tous comptes actifs
    "purchases_pending_count": 2
  },
  "cash_flow_7d": [
    { "date": "2026-04-14", "income_cents": 50000, "expense_cents": 12000 },
    ...
  ],
  "recent_transactions": [
    { "id": 42, "type": "income", "amount_cents": 50000, "description": "Facture INV-2026-0012",
      "transaction_date": "2026-04-14", "account": { "id": 1, "name": "Compte CPA" } },
    ...
  ]
}
```

8

GÉNÉRATION PDF FINANCIERS

DomPDF · templates Blade · envoi email · rapports

DomPDF est déjà dans le projet (utilisé pour les reçus de pointage). Le module Finance ajoute 4 nouveaux templates Blade : facture client, bon de commande, rapport mensuel, récapitulatif de dépenses. Aucune nouvelle dépendance.

Templates à créer

Fichier	Modèle	Données injectées	Usage
resources/views/finance/pdf/invoice.blade.php	Facture client professionnelle	invoice + items + company + logo	Envoi client + téléchargement
resources/views/finance/pdf/purchase-order.blade.php	Bon de commande fournisseur	purchase_order + items + supplier + company	Envoi fournisseur
resources/views/finance/pdf/monthly-report.blade.php	Rapport financier mensuel	kpis + expenses_by_category + invoices_list + transactions_list	Email manager
resources/views/finance/pdf/expense-summary.blade.php	Récapitulatif dépenses	expenses[] + totaux par catégorie + période	Export comptable

Numérotation automatique des factures

```
// app/Services/InvoiceNumberService.php
class InvoiceNumberService {
    public function generate(Company $company): string {
        return DB::transaction(function() use ($company) {
            // Lock exclusif sur la company pour éviter les doublons concurrents
            $company->lockForUpdate()->find($company->id);
            $prefix = $company->invoice_prefix ?? 'INV';
            $year = now()->year;
            $last = Invoice::where('company_id', $company->id)
                ->whereYear('created_at', $year)
                ->lockForUpdate()->count();
            return sprintf('%s-%d-%04d', $prefix, $year, $last + 1);
            // Résultat : INV-2026-0001, INV-2026-0002, ...
        });
    }
}
```

9

PLAN D'IMPLÉMENTATION — 65 TÂCHES

Sprint A → F · priorités · efforts · dépendances

Convention : Effort en jours-développeur (8h). Critique = bloquant. Haute = nécessaire pour la feature. Moyenne = qualité/UX. Les tâches sont ordonnées par dépendance logique.

SPRINT A Infrastructure & Base de données

· Semaine 1 · 14 tâches tâches · ~8,5 j-dev

ID	Tâche	Description	Effort	Priorité	Dépendances
FIN-01	9 migrations PostgreSQL	Créer les 9 migrations Laravel dans l'ordre : expense_categories, suppliers, finance_accounts, invoices, invoice_items, expenses, purchase_orders, purchase_items, finance_transactions. Vérifier les FK et l'ordre de création. Rollback testé.	1 j	Critique	—
FIN-02	Index de performance	Créer les 7 index listés en section 4.10. EXPLAIN ANALYZE sur les requêtes de dashboard et de listing pour confirmer < 5ms. Ajouter index composites si nécessaire après test.	0.25 j	Critique	FIN-01
FIN-03	Seeder plans finance	UpdatePlansFinanceFeatures : ajouter finance_enabled, finance_max_docs, finance_modules[] dans les features JSONB de Business et Enterprise. Seeder idempotent.	0.25 j	Critique	FIN-01
FIN-04	Modèle Invoice + InvoiceItem	Invoice.php : SoftDeletes, chiffrement notes sensibles, casts (dates, bigint vers centimes), scope company, relations hasMany(items), belongsTo(company), méthode isDraft(), isOverdue(), totalFormatted(). InvoiceItem.php : relation belongsTo(Invoice), totalComputed().	0.75 j	Critique	FIN-01
FIN-05	Modèle Expense + ExpenseCategory	Expense.php : SoftDeletes, scope company, relations category/supplier/account/createdBy/approvedBy, isPendingApproval(), méthode needsApproval(Company). ExpenseCategory.php : scope actif, relation hasMany(expenses).	0.5 j	Critique	FIN-01
FIN-06	Modèle Supplier	Supplier.php : chiffrement iban via Crypt, scope company, relation hasMany(purchaseOrders), lastOrderDate() helper.	0.25 j	Critique	FIN-01

ID	Tâche	Description	Effort	Priorité	Dépendances
FIN-07	Modèles PurchaseOrder + PurchaseItem	PurchaseOrder.php : SoftDeletes, numérotation, scope company, relations, méthodes de statut (isDraft, canSend, canConfirm, canReceive), totalReceived(). PurchaseItem.php : receivedPercentage().	0.5 j	Critique	FIN-01
FIN-08	Modèles FinanceAccount + FinanceTransaction	FinanceAccount.php : chiffrement account_number et iban, scope company, updateBalance(cents, type). FinanceTransaction.php : scope company + account, méthodes income/expense/transfer. Validation que balance_after_cents est toujours cohérent.	0.5 j	Critique	FIN-01
FIN-09	InvoiceNumberService	Service de génération de numéros séquentiels avec lock DB pour éviter les doublons concurrents. Format configurable : préfixe + année + 4 chiffres. Méthode generate(Company) → string.	0.5 j	Critique	FIN-04
FIN-10	FinancePdfService	Service DomPDF réutilisable pour les 4 templates. generateInvoice(Invoice), generatePurchaseOrder(PO), generateMonthlyReport(Company, year, month), generateExpenseSummary(Company, filters). Stockage dans storage/ structuré par company_id.	0.75 j	Critique	FIN-04
FIN-11	FinancePolicy + middleware	FinancePolicy : can('view-finance') → role finance ou manager:principal + plan.finance_enabled. CheckFinanceSubscription middleware : vérifie finance_enabled + finance_max_docs. Enregistrement dans Kernel.php alias 'finance.auth' et 'finance.subscription'.	0.5 j	Critique	FIN-04
FIN-12	FinanceTransactionObserver	Observer sur Invoice et Expense : après paiement facture → crée FinanceTransaction income. Après approbation dépense → crée FinanceTransaction expense. Met à jour FinanceAccount.current_balance_cents dans la même DB transaction.	0.75 j	Haute	FIN-08
FIN-13	Variables d'environnement et configuration	Ajouter dans .env : FINANCE_DEFAULT_TAX_RATE=19, FINANCE_RECEIPT_MAX_SIZE_MB=5, FINANCE_PDF_DPI=150, FINANCE_INVOICE_PREFIX=INV. Créer config/finance.php avec toutes les valeurs configurables.	0.25 j	Critique	—

ID	Tâche	Description	Effort	Priorité	Dépendances
FIN-14	Seeders données de démonstration	FinanceDemoSeeder : crée pour la company de démo 5 factures (statuts variés), 10 dépenses (catégories variées), 2 bons de commande, 3 fournisseurs, 2 comptes de trésorerie, 20 transactions. Pour les tests et la bêta.	0.5 j	Haute	FIN-03

SPRINT B API Backend — Factures & Dépenses

· Semaine 2 · 12 tâches tâches · ~10 j-dev

ID	Tâche	Description	Effort	Priorité	Dépendances
FIN-15	InvoiceController (CRUD + actions)	index() : liste paginée + filtres status/client/date. store() : crée invoice + items en DB transaction, génère numéro. show() : avec items + events. update() : seulement si draft. destroy() : soft delete. send() : FinancePdfService + SendInvoiceEmailJob. payment() : enregistre paiement + crée transaction. pdf() : download ou génère. duplicate().	2 j	Critique	FIN-04 FIN-09 FIN-10 FIN-11
FIN-16	Form Requests factures	CreateInvoiceRequest : client_name required, due_date after today, items required array min:1, items.*.unit_price_cents integer min:0, items.*.quantity numeric min:0.001. RecordPaymentRequest : amount_cents required integer, payment_method enum, account_id exists finance_accounts.	0.5 j	Haute	FIN-15
FIN-17	InvoiceResource + InvoiceCollection	InvoiceResource : inclure items, formatted amounts (DZD 1 250,00), status label en français, is_overdue flag, pdf_url présignée si pdf_path existe. InvoiceCollection : avec pagination meta et summary (total_pending_cents, count par statut).	0.5 j	Haute	FIN-15
FIN-18	SendInvoiceEmailJob	Job dispatchable en queue. Charge la facture + company + PDF. Envoie email via Mail::to(client_email)->send(new InvoiceEmail(invoice, pdf_path)). Template email Blade professionnel avec logo company. Gère retry (3 tentatives). Update invoice.sent_at.	0.75 j	Haute	FIN-10
FIN-19	ExpenseController	index() : liste + filtres. store() : crée expense, vérifie seuil approbation, crée transaction si auto-approved. show() : avec receipt_url présignée. update(). destroy(). uploadReceipt() : validation image, compression optionnelle, stockage. approve() : valide + crée transaction. reject() : avec motif.	1.5 j	Critique	FIN-05 FIN-11 FIN-12
FIN-20	Form Requests dépenses	CreateExpenseRequest : amount_cents integer min:1, category_id exists expense_categories, expense_date date not_in_future, payment_method enum. UploadReceiptRequest : receipt file mimes:jpeg,png,pdf max:5120.	0.5 j	Haute	FIN-19

ID	Tâche	Description	Effort	Priorité	Dépendances
FIN-21	ExpenseResource + ExpenseCategoryController	ExpenseResource : amount formaté, category avec couleur et icône, receipt_url présignée TTL 15min, can_approve flag selon rôle. ExpenseCategoryController : index/store/update/destroy. CategoryResource avec budget_monthly formaté.	0.5 j	Haute	FIN-19
FIN-22	Routes API V1 Finance	routes/api.php : groupe /finance avec middleware [sanctum, tenant, finance.auth, finance.subscription]. Toutes les routes factures, dépenses, catégories. Convention : api.v1.finance.invoices.* etc. Routes séparées pour upload-receipt (pas de finance.subscription car upload avant la création).	0.25 j	Critique	FIN-15 FIN-19
FIN-23	SupplierController	CRUD complet fournisseurs. index() avec filtre actif/inactif. show() inclut historique des 10 dernières commandes. Chiffrement iban dans store/update. Déchiffrement uniquement pour l'affichage (jamais dans les logs).	0.75 j	Haute	FIN-06
FIN-24	Tests Pest Sprint B	Tests : create invoice (succès + plan insuffisant + items vides), send invoice (mock email + statut), record payment (partiel + total), create expense (auto-approved + pending), upload receipt (type invalide + trop grand), approve expense (mauvais rôle), isolation tenant. Coverage > 85%.	2 j	Haute	FIN-15 FIN-19
FIN-25	Command calculat e-overdue-invoices	php artisan finance:update-overdue — passe en statut 'overdue' les factures dont due_date < today et statut = 'sent'. Planifier en Kernel::class::schedule() quotidiennement. Envoie notification push au manager pour chaque facture overdue.	0.5 j	Haute	FIN-04
FIN-26	Rate limiting endpoints finance	throttle(120, 1) sur les endpoints liste et détail. throttle(20, 1) sur upload-receipt. throttle(10, 1) sur send-email. Via RateLimiter::for() dans RouteServiceProvider.	0.25 j	Haute	FIN-22

SPRINT C API Backend — Achats, Trésorerie & Rapports

· Semaine 3 · 10 tâches tâches · ~8 j-dev

ID	Tâche	Description	Effort	Priorité	Dépendances
FIN-27	PurchaseOrderController	index/store/show/update/destroy. send() : génère PDF BC + email fournisseur. confirm() : statut=confirmed. receive() : corps {items: [{id, received_quantity}]} → met à jour received_quantity par item → si tout reçu : statut=received. Crée expense liée optionnellement.	1.5 j	Haute	FIN-06 FIN-07
FIN-28	PurchaseOrderResource + Form Requests	PurchaseOrderResource : inclut items avec received_percentage, supplier, formatted amounts, statut label, can_* flags pour les actions. CreatePurchaseOrderRequest : supplier_id exists + actif, expected_delivery nullable date after today, items array min:1.	0.5 j	Haute	FIN-27
FIN-29	FinanceAccountController	index() : liste comptes avec current_balance formaté. store() : crée compte avec initial_balance. show() : avec transactions récentes (n=10). update(). Les champs sensibles (iban, account_number) sont déchiffrés seulement pour l'affichage — jamais dans les logs.	0.75 j	Haute	FIN-08
FIN-30	FinanceTransactionController	index() par account_id : liste paginée avec filtres (type, date). store() : crée transaction manuelle (income ou expense) + met à jour balance en DB transaction. Validation : amount_cents > 0, account_id appartient à la company.	0.75 j	Haute	FIN-08 FIN-12
FIN-31	FinanceDashboardController	GET /finance/dashboard : calcule les 4 KPIs depuis les tables (pas de cache Phase 1), cash_flow_7d en agrégeant finance_transactions par jour, last 5 transactions. Tout en une seule requête SQL optimisée avec CTE si nécessaire. < 200ms.	1 j	Critique	FIN-08 FIN-04 FIN-05
FIN-32	FinanceReportController	monthly() : agrège factures + dépenses + transactions du mois. expenses_by_category avec % du total. pdf() : dispatch GenerateMonthlyReportJob → return {job_id}. Polling GET /finance/reports/{job_id}/status → {ready, pdf_url} si terminé. export() : CSV ou JSON selon le paramètre format.	1 j	Haute	FIN-10 FIN-08

ID	Tâche	Description	Effort	Priorité	Dépendances
FIN-33	GenerateMonthlyReportJob	Job async : charge données du mois, génère PDF via FinancePdfService, stocke, met à jour le statut en cache (Redis ou DB), envoie notification push Flutter 'Rapport prêt' avec lien téléchargement.	0.75 j	Haute	FIN-10 FIN-32
FIN-34	Routes Sprint C	Ajouter dans routes/api.php : /finance/purchase-orders/*, /finance/accounts/*, /finance/transactions/*, /finance/dashboard, /finance/reports/*. Vérifier que les routes respectent la convention de nommage api.v1.finance.*.	0.25 j	Critique	FIN-27 FIN-29 FIN-30 FIN-31
FIN-35	Tests Pest Sprint C	Tests : create PO (succès + fournisseur inactif), receive PO (partiel + complet), dashboard (KPIs corrects + isolation tenant), transaction manuelle (balance mise à jour), monthly report (agrégats corrects). Mock FinancePdfService dans les tests.	1.5 j	Haute	FIN-27 FIN-30 FIN-31
FIN-36	Audit log finance	FinanceAuditLogObserver sur Invoice, Expense, PurchaseOrder : enregistre chaque action (created, updated, status_changed, sent, paid) avec user_id, IP, before/after JSON diff. Table finance_audit_log. Endpoint GET /finance/audit-log/{entity_type}/{entity_id} pour le manager.	0.75 j	Haute	FIN-04 FIN-05 FIN-07

SPRINT D Dashboard Web Blade

· Semaine 4 · 10 tâches tâches · ~8,5 j-dev

ID	Tâche	Description	Effort	Priorité	Dépendances
FIN-37	Routes web finance (Blade)	routes/web.php : groupe /finance avec middleware [auth:web, finance.auth, finance.subscription]. Routes : dashboard, invoices (index/create/show/edit), expenses (index/create), purchase-orders (index/create/show), treasury (index), reports (index). Convention manager.finance.*.	0.25 j	Haute	FIN-11
FIN-38	WebFinanceDashboardController	Prépare les données pour la vue : KPIs (appel FinanceDashboardController logique), graphique cash flow, alertes (factures overdue, dépenses en attente). Injecte les données formatées dans la vue Blade. Utilise la même logique que l'API — réutiliser le service.	0.75 j	Haute	FIN-31 FIN-37
FIN-39	Vue finance/dashboard.blade.php	4 KPI cards avec couleurs ambre. Graphique cash flow (Alpine.js + Chart.js CDN). Table des factures en retard. Table des dépenses en attente d'approbation. Boutons d'action rapide : Nouvelle facture, Saisir dépense. Section alertes en rouge si factures overdue > 0.	1.5 j	Haute	FIN-38
FIN-40	Vues factures Blade (index/create/show)	invoice/index : tableau filtrable (Alpine.js), colonnes numéro/client/montant/statut(badge)/échéance/actions. invoice/create : formulaire avec gestion dynamique des lignes (Alpine.js, ajouter/supprimer, calcul HT/TVA/TTC temps réel). invoice/show : preview PDF-like + timeline événements + boutons d'action.	2 j	Haute	FIN-38
FIN-41	Vues dépenses Blade (index/create)	expense/index : liste groupée par mois avec totaux. Filtres catégorie (dropdown). Colonne reçu (icône si présent). expense/create : formulaire simple (montant, catégorie, description, date, mode paiement, upload photo). Validation JS côté client avant soumission.	1.5 j	Haute	FIN-38
FIN-42	Vues bons de commande Blade	purchase-orders/index : tableau avec statut stepper visuel (Tailwind timeline). purchase-orders/create : formulaire avec sélection fournisseur (AJAX autocomplete), lignes de commande dynamiques. purchase-orders/show : détail + progression livraison + boutons d'action selon statut.	1.5 j	Haute	FIN-38

ID	Tâche	Description	Effort	Priorité	Dépendances
FIN-43	Vue trésorerie Blade	treasury/index : liste des comptes avec soldes colorés. Graphique évolution par compte (Chart.js). Table transactions récentes filtrable. Formulaire quick-add transaction manuelle (Alpine.js modal). Bouton export CSV.	1 j	Haute	FIN-38
FIN-44	Vues rapports Blade	reports/index : sélecteur mois/année (2 selects). Rapport affiché en page : KPIs, dépenses par catégorie (graphique camembert Chart.js), liste factures du mois. Bouton Générer PDF (AJAX + polling status). Bouton Export CSV.	1 j	Haute	FIN-32 FIN-33
FIN-45	Composants Blade finance	x-finance-badge (invoice/expense/po statuts avec couleurs). x-amount-display (montant + devise formatés). x-finance-kpi (card KPI avec icône et tendance). x-invoice-item-row (ligne de facture éditable en Alpine.js). Tous réutilisables dans toutes les vues finance.	0.75 j	Haute	FIN-39
FIN-46	Menu sidebar finance	Ajouter le groupe 'Finance' dans config/navigation.php (FIN-DS via Design System). Sous-items : Dashboard, Factures, Dépenses, Achats, Trésorerie, Rapports. Couleur ambre. Visible si plan.finance_enabled. Badget numérique sur Factures si overdue > 0.	0.25 j	Critique	FIN-45

SPRINT E Application Flutter — 18 écrans

· Semaines 5-6 · 13 tâches tâches · ~14 j-dev

ID	Tâche	Description	Effort	Priorité	Dépendances
FIN-47	Dépendances Flutter finance	pubspec.yaml : ajouter fl_chart: ^0.69.x (graphiques), image_picker: ^1.x (photo reçu), image_compress: ^5.x (compression avant upload), intl: ^0.19.x (formatage devises). Vérifier compatibilité. Créer lib/features/finance/ avec sous-dossiers (models, repositories, blocs, screens, widgets).	0.5 j	Critique	—
FIN-48	Modèles Dart Finance	invoice_model.dart, invoice_item_model.dart, expense_model.dart, expense_category_model.dart, supplier_model.dart, purchase_order_model.dart, finance_account_model.dart, finance_transaction_model.dart, finance_dashboard_model.dart. Tous : fromJson/toJson, champs formatés (amountDisplay, statusLabel, statusColor).	1 j	Critique	FIN-47
FIN-49	FinanceRepository	lib/features/finance/repositories/finance_repository.dart : getDashboard(), getInvoices(filters), createInvoice(data), sendInvoice(id), recordPayment(id, data), getExpenses(filters), createExpense(data), uploadReceipt(file), approveExpense(id), getAccounts(), createTransaction(data), getMonthlyReport(year, month). Réutilise DioClient existant.	1.5 j	Critique	FIN-48
FIN-50	FinanceBloc + états	FinanceDashboardBloc, InvoiceBloc, ExpenseBloc, PurchaseBloc, TreasuryBloc. États : Loading, Loaded, Error, Submitting, Submitted. Events : Load, Refresh, Create, Update, StatusChange. Suivre le pattern BLoC existant du projet. FinanceNavigationCubit pour la navigation interne au module.	2 j	Critique	FIN-49
FIN-51	FinanceDashboard Screen + widgets	SCR-F01 : 4 FinanceMetricCard avec animations compteur (TweenAnimationBuilder). CashFlowChart (fl_chart LineChart) avec données 7j. RecentTransactionsList (5 items max). Pull-to-refresh. Skeleton screens pendant le chargement. Gestion état erreur (retry button).	1.5 j	Critique	FIN-50

ID	Tâche	Description	Effort	Priorité	Dépendances
FIN-52	Écrans Factures Flutter (SCR-F02 à F05)	InvoiceListScreen : liste + filtres chips + swipe actions + pull-to-refresh. InvoiceCreateScreen : formulaire multi-étapes avec gestion dynamique des lignes (ReorderableListView), calcul TTC temps réel, sélecteur TVA. InvoiceDetailScreen : preview + timeline + boutons. PaymentBottomSheet : montant, mode paiement, compte.	3 j	Critique	FIN-50
FIN-53	Écrans Dépenses Flutter (SCR-F06 à F08)	ExpenseListScreen : liste groupée par mois + barre progression budget + filtres. ExpenseCreateScreen : ultra-rapide (BigAmountInput en premier, CategoryPicker avec couleurs, CameraReceiptButton). ReceiptViewScreen : plein écran photo + InteractiveViewer + metadata.	2 j	Critique	FIN-50
FIN-54	BigAmountInput widget	Widget clavier numérique custom centré sur l'écran : grands chiffres, sélecteur devise inline, affichage formaté (1 250,00 DZD) pendant la saisie. Haptic feedback sur chaque touche. Utilisé dans ExpenseCreate, InvoiceItems, TransactionManuelle. Réutilisable.	0.75 j	Haute	FIN-47
FIN-55	CameraReceiptButton + upload flow	Widget : bouton avec icône caméra. Tap → ActionSheet (Prendre une photo / Galerie). Sélection → ImageCompressor (qualité 60%, max 1200px) → POST /finance/expenses/upload-receipt → receipt_path stocké en state. Feedback visuel : preview miniature + bouton supprimer. Gestion erreur upload.	0.75 j	Haute	FIN-49
FIN-56	Écrans Achats & Fournisseurs Flutter (SCR-F09 à F11)	PurchaseListScreen : liste avec statut stepper visuel. PurchaseCreateScreen : sélecteur fournisseur avec recherche, lignes d'achat. SupplierListScreen : annuaire avec search + swipe call/email. SupplierDetailScreen : infos + historique commandes.	2 j	Haute	FIN-50
FIN-57	Écrans Trésorerie & Rapports Flutter (SCR-F12 à F15)	TreasuryScreen : liste comptes + BalanceChart (fl_chart). TransactionListScreen par compte. TransactionCreateScreen. ReportsScreen : sélecteur mois + résumé + AsyncPdfButton (polling + notification quand prêt). Tous les graphiques en ambre pour la cohérence domaine.	2 j	Haute	FIN-50 FIN-33

ID	Tâche	Description	Effort	Priorité	Dépendances
FIN-58	FinanceNotificationsScreen + deep links (SCR-F17)	Écran notifications finance : liste triée par urgence (overdue en rouge en haut). Tap → context.go('/finance/invoices/{id}'). Badge rouge sur onglet Finance du BottomNav si notifications non lues. Persistance en local storage du dernier vu.	0.75 j	Haute	FIN-51
FIN-59	Tests widget Flutter finance	Tests : FinanceDashboardScreen (skeleton + loaded + error), InvoiceCreateScreen (calcul TTC temps réel), ExpenseCreateScreen (seuil approbation affiché), BigAmountInput (saisie + format), CameraReceiptButton (mock upload). FinanceBloc unit tests (transitions d'états). Coverage > 70%.	1.5 j	Haute	FIN-51 FIN-52 FIN-53

SPRINT F Tests, Sécurité & Déploiement

· Semaine 7 · 6 tâches tâches · ~5 j-dev

ID	Tâche	Description	Effort	Priorité	Dépendances
FIN-60	Tests E2E golden path complet	Scénario 1 (facture) : gestionnaire crée facture → envoi → manager voit notification → marque payée → trésorerie mise à jour. Scénario 2 (dépense) : gestionnaire saisit dépense > seuil + photo → manager reçoit notification → approuve → balance débité. Scénario 3 (achat) : BC créé → envoyé fournisseur → réception confirmée. Documenter avec screenshots.	1 j	Haute	Tout
FIN-61	Audit sécurité isolation tenant	Tests : accès facture company B avec token company A (404), montants négatifs acceptés (doit échouer 422), IBAN en clair dans les logs (grep automatique), balance_after_cents cohérence après N transactions simultanées (load test PostgreSQL), access finance sans rôle finance (403).	0.75 j	Critique	FIN-22 FIN-34
FIN-62	Test de charge API finance	k6 : simuler 50 utilisateurs créant simultanément des factures. Vérifier unicité des numéros INV (0 doublon toléré). Simuler 100 req/s sur /finance/dashboard. Objectif < 300ms p95. Ajuster index si nécessaire. Publier rapport de test.	0.5 j	Haute	FIN-02 FIN-31
FIN-63	Documentation API finance	Ajouter le module finance dans le fichier docs/dossierdeConception/01_API_CONTRATS_COMPLETS/02_API_CONTRATS_COMPLET.md existant. Tous les endpoints, schemas, erreurs. Mettre à jour l'ERD (03_ERD_COMPLET.md) avec les 9 nouvelles tables.	1 j	Haute	Tout
FIN-64	CI/CD — pipeline tests finance	Ajouter job 'finance-tests' dans tests.yml GitHub Actions. Base de données de test dédiée avec migrations finance. Linting Blade templates finance. Coverage report uploadé sur Codecov. Badge vert obligatoire avant merge sur main.	0.5 j	Haute	FIN-24 FIN-35 FIN-59
FIN-65	Mise à jour PILOTAGE.md et CHANGELOG	Ajouter Module Finance dans PILOTAGE.md scope livré. Mettre à jour la section EXCLUDED (retirer Finance). Ajouter dans CHANGELOG : tous les nouveaux endpoints, nouvelles tables, nouveau rôle gestionnaire financier. Incrémenter PROGRAM_VERSION.	0.25 j	Haute	Tout

10

TESTS — STRATÉGIE COMPLÈTE

Coverage · cas critiques · E2E · sécurité financière

Couche	Framework	Coverage cible	Cas critiques à couvrir
Models Eloquent	Pest + RefreshDB	100%	Calcul totaux, méthodes statut, chiffrage IBAN, unicité numéros
API Controllers	Pest + Feature	85%+	Auth, plan limits, isolation tenant, validation montants
Policies	Pest	100%	Gestionnaire finance vs manager vs employé sur chaque action
Jobs & Observers	Pest + Queue fake	90%	Observer balance mise à jour, email job, PDF job
Dashboard Blade	Laravel Dusk	Flux critiques	Créer facture, saisir dépense, voir trésorerie
Flutter Blocs	flutter_test	80%+	Transitions d'états, erreurs réseau, cas limites
Flutter Widgets	flutter_test	70%+	BigAmountInput, CameraButton, skeleton, graphiques
E2E Flutter+API	Manuel + script	3 scénarios golden	Facture complète, Dépense+approbation, Achat+réception
Sécurité	Tests manuels	Tous vecteurs	Isolation tenant, montants négatifs, IBAN logs, concurrence
Charge	k6	50 users simultan.	Unicité numéros, dashboard < 300ms p95

Cas de test critique — unicité numéros de facture sous charge

```
// tests/Feature/Finance/InvoiceNumberUniquenessTest.php
it('generates unique invoice numbers under concurrent load', function () {
    $company = Company::factory()->withFinancePlan()->create();
    $manager = Employee::factory()->financeManager()->for($company)->create();

    // Simuler 20 requêtes simultanées via Promise-like
    $results = collect(range(1, 20))->map(fn() =>
        actingAs($manager)->postJson('/api/v1/finance/invoices', invoicePayload())
    );

    $numbers = Invoice::where('company_id', $company->id)->pluck('number');
    expect($numbers->count())->toBe(20) // 20 créées
        ->and($numbers->unique()->count())->toBe(20); // toutes uniques
});
```

11

ROADMAP POST-PHASE 1

Phases 2 → 5 spécifiées avec tâches — à implémenter après la Phase 1

Ces features sont spécifiées maintenant pour guider les choix d'architecture en Phase 1 (ex: colonnes extensibles en JSONB, hooks pour la réconciliation bancaire, structure prête pour l'IA). Elles ne sont pas à implémenter maintenant.

Phase 2 — Analytics avancés & Tableaux de bord IA

ID	Tâche	Description	Effort	Priorité	Dépendances
P2-F01	Prévision de trésorerie IA	Modèle de régression simple (scikit-learn via microservice Python) entraîné sur l'historique transactions. Prédit le solde à 30/60/90 jours. Affiché dans TreasuryScreen avec intervalle de confiance. Alerte si solde prévu < seuil configuré.	8 j	Haute	FIN-08
P2-F02	Détection anomalies dépenses	Règles + ML : dépense hors catégorie habituelle, montant > 3σ de la moyenne, doublon suspect (même montant + même jour). Notification manager + flag 'suspicious' sur la dépense. Dashboard anomalies.	5 j	Haute	FIN-05
P2-F03	Dashboard analytique avancé	Nouveaux écrans : Analyse rentabilité par client (factures émises vs temps passé si couplé au module RH), Top 5 fournisseurs par volume, Évolution CA mensuel sur 12 mois, Budget vs Réel par catégorie avec drill-down.	5 j	Haute	P2-F01
P2-F04	Objectifs et budgets	Le manager définit des budgets mensuels par catégorie et un objectif de CA. Barres de progression avec alertes à 80% et 100% du budget. Comparaison mois/mois et année/année.	3 j	Haute	FIN-05
P2-F05	Rapport automatique hebdomadaire	Email hebdomadaire (lundi 8h) au manager et au gestionnaire : résumé semaine précédente, 3 points d'attention (factures overdue, dépenses anormales, solde bas), prévision semaine suivante.	2 j	Moyenne	P2-F01

Phase 3 — Intégration bancaire & rapprochement automatique

ID	Tâche	Description	Effort	Priorité	Dépendances
P3-F01	Import relevé bancaire (CSV/OFX)	Upload d'un fichier CSV ou OFX exporté depuis la banque. Parser les transactions. Interface de rapprochement : lier chaque ligne bancaire à une facture payée ou une dépense. Score de correspondance automatique (montant + date $\pm$ 3j).	6 j	Haute	FIN-08
P3-F02	Connexion bancaire directe (Open Banking)	Intégration Bridge API (France) ou Fintecture (MENA) pour import automatique des transactions bancaires. Authentification OAuth2 de la banque. Synchronisation automatique quotidienne. Compatible avec CIH Bank, CPA, BNP Paribas.	12 j	Haute	P3-F01
P3-F03	Rapprochement automatique IA	Modèle de classification pour matcher automatiquement les transactions bancaires avec les factures et dépenses Leopardo RH. Taux de rapprochement automatique cible : > 85%. Les 15% restants présentés à l'utilisateur pour validation manuelle.	8 j	Haute	P3-F02
P3-F04	Export comptable (SAGE, ERP, CSV normalisé)	Export des écritures comptables dans les formats : CSV Plan Comptable Général (PCG), XML SAGE, JSON générique. Paramétrage des comptes comptables par catégorie de dépense. Compatible avec la plupart des cabinets comptables.	4 j	Haute	FIN-32

Phase 4 — Module Paie intégrée complète

ID	Tâche	Description	Effort	Priorité	Dépendances
P4-F01	Calcul de paie automatique (couplage RH+Finance)	Utilise les données de pointage du module RH (heures travaillées, absences, retards) pour calculer automatiquement la paie brute. Applique les règles du pays (cotisations CNAS/CNSS, IR, IR forfaitaire). Génère bulletin de paie PDF.	15 j	Critique	FIN-10
P4-F02	Virement masse (export bancaire)	Génère le fichier de virement de masse pour toute la paie du mois : format SEPA XML ISO 20022 (Europe), format spécifique CIH/Attijariwafa (Maroc), CPA (Algérie). Envoi direct via Open Banking Phase 3.	6 j	Haute	P4-F01 P3-F02
P4-F03	Avances sur salaire intégrées	Les avances saisies dans le module RH sont automatiquement déduites du salaire calculé. Traçabilité complète avance → remboursement → bulletin de paie. Notification employé lors de la déduction.	3 j	Haute	P4-F01

ID	Tâche	Description	Effort	Priorité	Dépendances
P4-F04	Déclarations sociales & fiscales	Génère les déclarations périodiques : DAS (Déclaration Annuelle des Salaires), G50 mensuelle (Algérie), CNSS mensuelle (Maroc). Export PDF + transmission électronique si API disponible.	8 j	Haute	P4-F01

Phase 5 — Écosystème & API publique Finance

ID	Tâche	Description	Effort	Priorité	Dépendances
P5-F01	API publique finance (plan Enterprise)	Exposer les endpoints finance via l'API publique de Leopard RH (déjà prévue dans le plan Enterprise). Webhook configurable : facture payée, dépense approuvée, solde bas. Authentification API Key. Documentation OpenAPI générée automatiquement.	4 j	Haute	FIN-22
P5-F02	Intégration e-commerce (WooCommerce, Shopify)	Plugin qui synchronise automatiquement les commandes e-commerce vers des factures Leopard RH. Mapping produit → ligne de facture. Stock synchronisé avec les bons de commande fournisseurs.	8 j	Moyenne	P5-F01
P5-F03	Mode multi-sociétés	Un manager peut gérer les finances de plusieurs entités légales depuis la même app. Consolidation des bilans. Transferts inter-sociétés. Rapport consolidé de groupe.	10 j	Haute	FIN-31
P5-F04	Facturation récurrente	Créer un modèle de facture récurrente (abonnement) : fréquence (mensuelle, trimestrielle, annuelle), date de génération automatique, envoi automatique au client. Gestion des échecs d'envoi.	3 j	Haute	FIN-15
P5-F05	Portail client en ligne	Lien public permettant au client de consulter ses factures, télécharger les PDFs, et enregistrer un paiement en ligne (intégration Stripe/Fintecture). Personnalisable avec le logo de l'entreprise.	6 j	Haute	FIN-15 P3-F02

12

DÉFINITION OF DONE — 18 CRITÈRES

Critères vérifiables avant mise en production du module Finance

ID	Catégorie	Critère de done
FIN-DOD-01	FONCTIONNEL	Le gestionnaire financier peut créer, modifier et envoyer une facture client par email avec PDF depuis l'app Flutter.
FIN-DOD-02	FONCTIONNEL	Le gestionnaire peut saisir une dépense avec photo de reçu en moins de 5 taps depuis l'app mobile.
FIN-DOD-03	FONCTIONNEL	Une dépense supérieure au seuil configuré est automatiquement mise en statut 'En attente d'approbation' et notifie le manager.
FIN-DOD-04	FONCTIONNEL	Le gestionnaire peut créer et envoyer un bon de commande PDF à un fournisseur depuis l'app.
FIN-DOD-05	FONCTIONNEL	Le manager voit en temps réel le solde de trésorerie consolidé de tous ses comptes depuis l'app mobile.
FIN-DOD-06	FONCTIONNEL	Un rapport mensuel complet peut être généré en PDF et reçu par email depuis le dashboard.
FIN-DOD-07	FONCTIONNEL	La numérotation des factures est séquentielle et unique — zéro doublon même sous charge concurrent.
FIN-DOD-08	RÔLES	Le gestionnaire financier n'a accès à aucune donnée RH (salaires, pointages, absences). Vérification par test automatique.
FIN-DOD-09	RÔLES	Un employé standard ne peut pas accéder au module finance — retour 403 sur tous les endpoints.
FIN-DOD-10	SÉCURITÉ	L'isolation multitenancy est vérifiée : aucune donnée financière d'une entreprise n'est visible d'une autre.
FIN-DOD-11	SÉCURITÉ	Les IBAN et numéros de compte ne sont jamais en clair dans les logs applicatifs — vérification grep automatique.
FIN-DOD-12	SÉCURITÉ	La cohérence balance_after_cents est garantie après 100 transactions simultanées simulées par le test de charge.
FIN-DOD-13	PLAN	Un manager Plan Business peut utiliser factures et dépenses. Un manager Starter voit un écran d'upgrade propre.
FIN-DOD-14	PERFORMANCE	GET /finance/dashboard retourne en < 300ms p95 sous 50 utilisateurs simultanés.
FIN-DOD-15	PERFORMANCE	La liste factures se charge en < 1s sur l'app Flutter en 4G standard.
FIN-DOD-16	QUALITÉ	Coverage tests Pest > 85% sur tous les controllers et policies du module finance.
FIN-DOD-17	COULEUR	Toute l'interface finance utilise exclusivement la couleur ambre (#D97706) comme couleur de domaine — aucun vert RH dans le module finance.

ID	Catégorie	Critère de done
FIN-DOD-18	DOCUMENTATION	PILOTAGE.md et ERD mis à jour. Guide gestionnaire financier (in-app) livré. Endpoints documentés dans 02_API_CONTRATS_COMPLET.md.

Récapitulatif global des efforts

Sprint	Contenu	Semaine	Tâches	Effort	Livrable
A — Infrastructure	9 migrations + 8 modèles + services clés	Sem. 1	14	~8,5 j	Base technique prête
B — API Factures & Dépenses	Controllers + jobs + tests	Sem. 2	12	~10 j	API documents financiers
C — API Achats & Trésorerie	PO + comptes + dashboard + rapports	Sem. 3	10	~8 j	API trésorerie complète
D — Dashboard Web Blade	6 vues + composants Blade	Sem. 4	10	~8,5 j	Interface web finance
E — Application Flutter	18 écrans + blocs + widgets	Sem. 5-6	13	~14 j	App mobile complète
F — Tests & Déploiement	E2E + sécurité + charge + docs	Sem. 7	6	~5 j	Module production-ready
TOTAL	Module Finance complet Phase 1	7 semaines	65	~54 j	Feature enterprise-grade

Ce document est la référence complète pour l'implémentation du Module Finance & Gestion de Leopardo RH. Version 1.0 — Avril 2026 — Équipe Produit Leopardo RH. Toute question d'implémentation non couverte ici doit être documentée et ajoutée avant démarrage du sprint concerné.